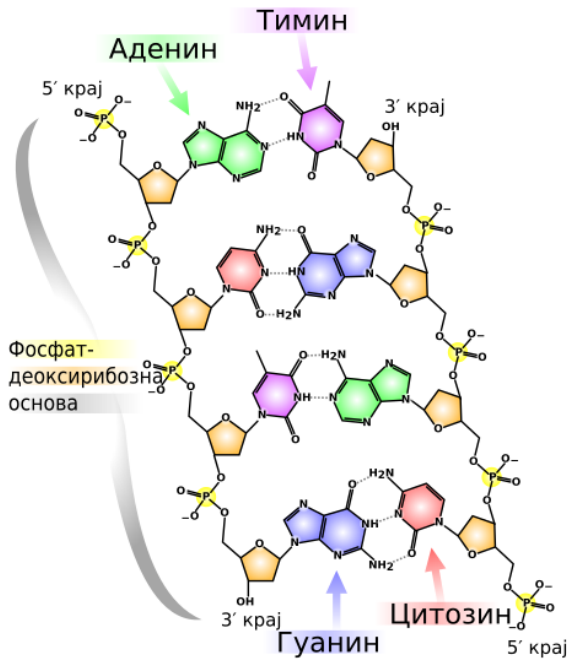


# Упутство и спецификација

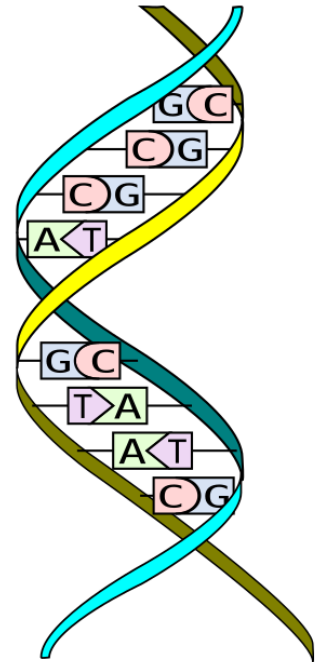
- Да би се открило присуство COVID-19 вируса као и за потребе проналажења лека за исти, потребно је урадити секвенцирање његовог дезоксирибонуклеинске киселине тј. ДНК. Секвенцу ДНК чине 4 азотне базе:

- Аденин
- Цитозин
- Гуанин
- Тимин

Означавају се скраћено са првим словом из назива: А, С, G, Т. Слика 1а приказује хемијски изглед ДНК ланца са 4 базе.



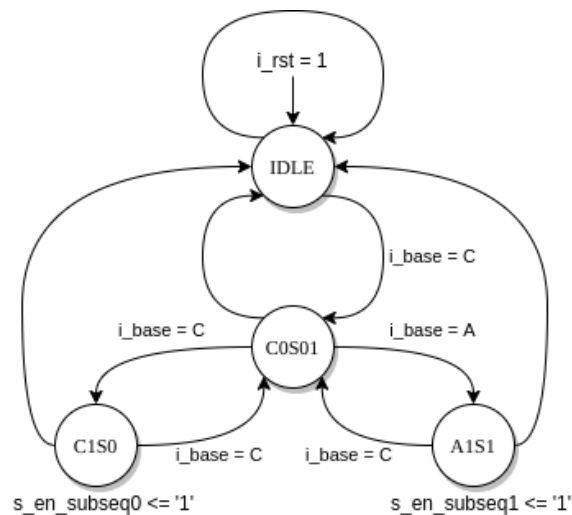
(а) Хемијска структура ДНК



(б) ДНК секвенца

Слика 1: Изглед ДНК

На Слици 1б је пример једне ДНК секвенце. Ако пратимо ланац са плавкастом траком који почиње из горњег десног ћошка слике секвенца је GCCACATG. У овој секвенци се даље могу тражити подсеквенце. Тако на пример, ако се траже подсеквенце CC и CA, биће пронађена једна CC и две CA. Проналажење ове две подсеквенце се може извести путем аутомата. На улазе  $i\_base$  се доводи секвенца GCCACATG, по једна база у сваком такту, а стања прате подсеквенце.



Слика 2: Аутомат претраге ДНК секвенце

Слика 2 приказује аутомат за горепоменуће подсеквенце СС и СА. На почетку се креће из `IDLE` стања. Обе подсеквенце почињу истим стањем `COS01`. Овде `C` означава базу, `0` иза њега означава редни број у подсеквенци, `S12` означава да се исто стање користи у претраги `0.` и `1.` подсеквенце, односно СС и СА. После се врши грађање у два стања: `C1S0` за `0.` подсеквенцу СС и `A1S1` за `1.` подсеквенцу СА. У излазној функцији, два горепоменућа стања ће биће постављени одговарајући сигнали дозволе бројања, тако да ће бројачи подсеквенци моћи избројати појаве одговарајуће секвенце. У случају новог `C` одмах се покреће нова претрага, док у случају појаве неке базе ван тражених подсеквенци, претрага почиње из почетка из `IDLE` стања.

- Спецификација је следећа:

1. Направити аутомат са коначним бројем стања за проналажење подсеквенци:
  - Аутомат треба да детектује следеће подсеквенце:
    - (a) `CAC`
    - (b) `CAT`
    - (v) `CGA`
  - Сигнал стања аутомата назвати `s_state`, а следећег стања `s_next_state`.
  - Као улаз у аутомат користити `s_base`.
  - Имена стања дефинисати по узору на пример дат горе.
  - Користити заједничка стања за заједничке делове подсеквенци.
  - У излазној функцији, крајњим стањима подсеквенце активирати један од сигнала дозволе бројања ( `s_en_subseq0`, `s_en_subseq1` или `s_en_subseq2` ), како би одговарајући бројач избројао подсеквенцу.
  - Излазна и прелазна функција треба бити комбинациона. Никако не реализовати прелазну функцију у секвенцијалном процесу.
  - Користити синхрони ресет.
2. Направити бројач `0.` подсеквенце:
  - Сигнал бројача назвати `s_cnt_subseq0`.
  - Користити `s_en_subseq0` као сигнал дозволе бројања.
  - Бројати по модулу `9`.
  - Број бита је `4`.
  - Користити асинхрони ресет.
3. Направити бројач `1.` подсеквенце:
  - Сигнал бројача назвати `s_cnt_subseq1`.
  - Користити `s_en_subseq1` као сигнал дозволе бројања.
  - Бројати по модулу `4`.
  - Број бита је `4`.
  - Користити асинхрони ресет.
4. Направити бројач `2.` подсеквенце:
  - Сигнал бројача назвати `s_cnt_subseq2`.
  - Користити `s_en_subseq2` као сигнал дозволе бројања.
  - Бројати по модулу `8`.
  - Број бита је `4`.
  - Користити асинхрони ресет.
5. Направити излазни мултиплексер за селекцију излаза са бројача:
  - Реализовати мултиплексер преко `process-if`.
  - Користити `i_cnt_subseq_sel` сигнал као селекциони.
  - Користити `o_cnt_subseq` као излаз из мултиплексера.
  - Улази ће наравно бити `s_cnt_subseq0...`
6. Направити улазни мултиплексер за селекцију базе која ће ући у аутомат:
  - Реализовати мултиплексер преко `process-case`.
  - Користити `i_base_src_sel` сигнал као селекциони.
    - \* Ако је селекциони улаз `'0'` селекувати `i_base`.
    - \* Ако је селекциони улаз `'1'` селекувати `s_sh_base`.
  - Користити `s_base` као излаз из мултиплексера.
7. Направити померачки регистар који ће једну по једну базу из секвенце послати на аутомат:
  - Реализовати комбинациону мрежу померачког регистара преко `process-case`.
  - Користити `s_sh_reg` као сигнал регистара.

- \* Када је `i_load_sequence` на 1 уписати `i_sequence` у регистар.
  - \* У противном померати регистар улево.
  - За излаз `s_sh_base` узимати из `s_sh_reg` 2 бита највеће вредности.
  - Користити асинхрони ресет.
- На основу спецификације нацртати дијаграм стања аутомата. Дијаграм назвати `FSM.png` или `FSM.jpg`, који је већ формат слике у питању, и ставити га у `LPRS1_Homework3_RA_121_2021_Solution` фолдер.
  - Реализовати систем у VHDL-у по горепоменутој спецификацији и дијаграму стања аутомата. Реализацију урадити у `LPRS1_Homework3_RA_121_2021_Solution/lprs1_homework3.vhd` фајлу.
    - Кодовање база у VHDL-у је дефинисано константама.
    - Изнад кода сваке компоненте у коментару написати име описане компоненте.
  - У тестбенч фајлу `LPRS1_Homework3_RA_121_2021_Solution/lprs1_homework3.vhd` верификовати систем по следећим тачкама:
    1. На улаз `i_base` довести следеће секвенце:
      - (a) `CTGT CATG CAGC TAAG GTGG CGCC TTAG GGTC`
      - (б) `CAAG TATC TTCA TCTC CAAG ATGA GAAG TAGA`
    2. Мењати све улазне сигнале (укључујући и `i_rst`) на опадајућу ивицу такта.
    3. 0. секвенцу слати базу по базу на `i_base` улаз.
      - (a) За сваку базу држати на улазу `i_base` у дужини једне периоде такта.
      - (б) Не заборавити селектовати одговарајућу извор базе са `i_base_src_sel`.
    4. 1. секвенцу слати базу по базу на `i_base` улаз.
      - (a) Поставити секвенцу на `i_sequence` и учитати је активацијом `i_load_sequence`.
      - (б) Гледајући у интерне сигнале одредити колико је периода такта неопходно потребно да би се свака база секвенце послала на аутомат.
      - (в) Не заборавити селектовати одговарајућу извор базе са `i_base_src_sel`.
    5. Пре сваке секвенце у коментару написати секвенцу.
    6. Пре сваке секвенце држати ресет активан 14 тактова.
    7. Након краја секвенце оставити сачекати 1 такт, како би се бројач инкрементирао.
    8. Ишчитати вредности бројача излазног мултиплексера на `o_cnt_subseq` излазу, по једна вредност `i_cnt_subseq_sel` по периоди такта.
    9. На крају оставити систем у ресету.
  - На крају, зиповати фолдер `LPRS1_Homework3_RA_121_2021_Solution` у зип фајл `LPRS1_Homework3_RA_121_2021_Solution.zip`. Не треба зиповати фајлове из `LPRS1_Homework3_RA_121_2021_Solution`, већ баш фолдер са фајловима. Не треба зиповати фолдер целог пројекта, него само `LPRS1_Homework3_RA_121_2021_Solution` фолдер. Ако се пошаље нешто друго, рецимо цео пројекат, рад неће бити гледан и резултоваће са оценом **D** односно **0** бодова.
  - У договору са својим асистентном преко чета у MS Teams-у или мејлом, послати као решење горепоменути зип фајл.